# Python and numpy for Assignment 4

Step-by-step guide, grows with the project

Filippo Ferroni

Version 1.1, 25 October 2026 — Lesson 1: functions and linear algebra

**How to use this guide.** Each lesson matches one piece of the project. You learn only the Python you need, when you need it. Each lesson ends with exercises; solutions to the project TODOs are added *after* you have written them yourself.

## Contents

**1 Project map** **1**

**2 Running a Python file** **1**

**3 Variables and `print`** **1**

**4 Functions** **2**

**5 numpy: vectors and matrices** **2**

**6 Operations: element-wise vs matrix product** **2**

**7 From formula to code: the 4-step method** **3**

7.1 Worked example: ridge (TODO 2) . . . . . 3

**8 Common errors and what they mean** **3**

**9 Lesson 1 exercises** **4**

## 1 Project map

| File | What it does | Lesson |
|---|---|---|
| `src/models.py` | least squares, ridge, mean squared error | 1 (this one) |
| `src/data.py` | generates synthetic data, loads the real dataset, train/test split | 2 |
| `src/experiment.py` | loop over $d$: train and measure the errors | 3 |
| `src/plots.py` | plots with matplotlib | 4 |

## 2 Running a Python file

- A `.py` file is text with instructions executed top to bottom.
- From the terminal, inside the repo folder:

```
python src/models.py
```

(on a Mac the command is sometimes `python3`).

- In VS Code: open the `smml-double-descent` folder, open the file, then use the "Run" button at the top right.
- Lines starting with `#` are **comments**: Python ignores them, they are for the reader.

## 3 Variables and `print`

```
m = 3               # an integer
alpha = 0.1         # a decimal number (use a dot, not a comma)
name = "ridge"      # a string (text)
print("alpha is", alpha)
```

The `=` sign **assigns** a value to a name; it is not a mathematical equality.

## 4 Functions

A function is a recipe: it takes inputs, does some computation, returns a result.

```
def square(x):
    """Return x squared."""   # docstring: explains what it does
    result = x ** 2           # ** is the power operator
    return result

print(square(4))   # prints 16
```

- `def` defines the function; the **arguments** (inputs) go in parentheses.
- The body is **indented by 4 spaces**. Indentation tells Python which lines belong to the function. If it is wrong: `IndentationError`.
- `return` gives back the result. Without `return`, the function returns `None` ("nothing").
- `raise NotImplementedError` means "not written yet": you find it in the TODOs and replace it with your code.

**The `if __name__ == "__main__":` block.** The code below this line runs only when you launch the file directly (`python src/models.py`), not when another file does `import` of `models.py`. We use it for quick checks.

## 5 numpy: vectors and matrices

```
import numpy as np        # import numpy under the short name np

y = np.array([1.0, 2.0, 3.0])            # vector with 3 entries
X = np.array([[1.0, 0.0],
              [1.0, 1.0],
              [1.0, 2.0]])               # 3 x 2 matrix (3 rows, 2 columns)

print(X.shape)       # (3, 2)
print(X.shape[0])    # 3 = number of rows = m
print(X.shape[1])    # 2 = number of columns = d
print(X[0])          # first row: [1. 0.]   (counting starts at 0!)
print(X[2, 1])       # row 3, column 2: 2.0
```

**Careful:** Python counts from 0. The first element is `[0]`, not `[1]`.

# 6 Operations: element-wise vs matrix product

| Maths | numpy | Notes |
|---|---|---|
| $A + B$ | `A + B` | element-wise, same shapes |
| $\alpha A$ | `alpha * A` | multiplies every entry by $\alpha$ |
| $v_i^2$ for every $i$ | `v ** 2` | element-wise power |
| $AB$ (matrix product) | `A @ B` | **not** `A * B`! |
| $A^\top$ | `A.T` | transpose |
| $I_d$ | `np.eye(d)` | $d \times d$ identity |
| $\frac{1}{m}\sum_i v_i$ | `np.mean(v)` | mean of the entries |
| $\|w\|$ | `np.linalg.norm(w)` | Euclidean norm |
| $A^+$ | `np.linalg.pinv(A)` | pseudo-inverse |
| solve $Aw = b$ | `np.linalg.solve(A, b)` | better than computing $A^{-1}$ |

**Classic mistake:** `A * B` multiplies entry by entry, it is *not* the row-by-column product. The formulas in the slides almost always need `@`.

**The shape rule.** `A @ B` works only if the number of columns of `A` equals the number of rows of `B`: $(p \times q) \,@\, (q \times r) = (p \times r)$. Otherwise numpy raises an error containing the word `mismatch`. **Before writing a line, check the shapes by hand.**

# 7 From formula to code: the 4-step method

1. Write down the mathematical formula.
2. Give a name to each piece (e.g. $A$, $b$).
3. Translate each symbol using the table in the previous section.
4. Return the result with `return`.

## 7.1 Worked example: ridge (TODO 2)

Formula (slides 9, p. 5): $(\alpha I + X^\top X)\, w = X^\top y$, i.e. $Aw = b$ with $A = \alpha I + X^\top X$, $b = X^\top y$.

```
def fit_ridge(X, y, alpha):
    d = X.shape[1]                      # number of features (columns of X)
    A = alpha * np.eye(d) + X.T @ X     # d x d matrix
    b = X.T @ y                         # vector with d entries
    w = np.linalg.solve(A, b)           # solves A w = b
    return w
```

Shape check with $X$ of shape $m \times d$ and $y$ with $m$ entries:

- `np.eye(d)`: $d \times d$; `X.T @ X`: $(d \times m)(m \times d) = d \times d$; sum: $d \times d$. ✓
- `X.T @ y`: $(d \times m)(m) = d$. ✓
- `solve`: from a $d \times d$ matrix and a $d$-vector it returns $w$ with $d$ entries. ✓

## 8 Common errors and what they mean

| Message | Likely cause |
|---|---|
| `IndentationError` | wrong indentation (use 4 spaces, do not mix tabs and spaces) |
| `NameError: name 'np' is not defined` | missing `import numpy as np` at the top of the file |
| `NotImplementedError` | you left the `raise NotImplementedError` line in the TODO |
| `ValueError: matmul: ... mismatch` | incompatible shapes in a `@`: check the shapes |
| `LinAlgError: Singular matrix` | `solve` with a non-invertible matrix (e.g. ridge with $\alpha = 0$ and $d > m$) |
| `ModuleNotFoundError: numpy` | numpy not installed: `pip install -r requirements.txt` |

## 9 Lesson 1 exercises

1. With $X$ of shape $3 \times 2$: why does `X @ X` fail while `X.T @ X` works? What is the shape of the result?
2. What is the difference between `X.T @ X` and `X @ X.T`? What are their shapes with $m = 3$, $d = 2$? (Hint: one of them appears in the minimum-norm formula, theory Sec. 4.3.)
3. What does `np.array([1.0, -2.0, 3.0]) ** 2` print? And `np.mean` of that result?
4. **TODO 1** in `models.py`: translate $w = X^{+}y$ with the 4-step method.
5. **TODO 3** in `models.py`: translate $\frac{1}{m}\|Xw - y\|^2$. (Hint: first the error vector, then the mean of the squares.)

### Solutions (exercises 1–3)

1. `X @ X` needs columns of $X$ (2) = rows of $X$ (3): false, hence the error. `X.T @ X` is $(2 \times 3)(3 \times 2) = 2 \times 2$.
2. `X.T @ X` is $d \times d = 2 \times 2$ (normal equations, ridge); `X @ X.T` is $m \times m = 3 \times 3$ (it appears in $w^{+} = X^{\top}(XX^{\top})^{-1}y$).
3. `[1. 4. 9.]`; the mean is $14/3 \approx 4.67$.

### TODO solutions (written by you, checked on 25/10)

```python
def fit_least_squares(X, y):
    w = np.linalg.pinv(X) @ y          # w = X^+ y, minimum norm for every d
    return w

def mse(X, y, w):
    y_hat = X @ w                      # predictions on the set (m values)
    error = np.mean((y_hat - y) ** 2)  # mean of the squared errors
    return error
```

Check outputs: A = [1, 1], B = [1, 10], C ≈ [1.0, 0.01], D ≈ $2 \cdot 10^{-32}$ (numerical zero: the computer works with about 16 significant digits, so such a tiny value is a zero). **Lesson from check C:** on the same data the minimum-norm solution puts a weight of 10, ridge 0.01: this is the mechanism that removes the peak, seen on your own code.